

# FlowHub - Reflexion: KI-gestützte Entwicklung

CAS AI-Assisted Software Engineering (AISE) · W4B-C-AS001 · ZH-Sa-1 · FS26 **Student:** Andreas Imboden · **Stand:** Juni 2026

Dieses Dokument reflektiert den **KI-Einsatz** über alle fünf CAS-Blöcke: welche Werkzeuge wie eingesetzt wurden, wie sich der Workflow entwickelt hat, was verlässlich funktioniert hat und was nicht — und welche Disziplinen als tragfähig über das Projekt hinaus erscheinen. Die detaillierte, blockweise Aufschlüsselung (generated-vs-handwritten-Anteile, Korrektur-Geschichten) liegt in `docs/ai-usage.md`.

## 1. KI-Werkzeuge im Entwicklungsprozess

| Werkzeug                                              | Einsatzbereich                                                                                                                                                                  |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude Code (Opus, 1M-Kontext)                        | Brainstorming, Spec-/Plan-Erstellung, ADR-Drafts, Controller für Subagent-Dispatches                                                                                            |
| Claude Sonnet (Subagents)                             | Implementer + Spec-Reviewer + Code-Quality-Reviewer unter dem <code>subagent-driven-development</code> - Workflow                                                               |
| Claude Haiku (Subagents)                              | Mechanische Tasks (csproj/Markdown/YAML), ca. 60 % Token-Ersparnis gegenüber Sonnet                                                                                             |
| <code>/ultrareview</code> (Multi-Agent-Branch-Review) | Architektonisches Review über einen ganzen Feature-Branch — fängt System-Invarianten ab, die Per-Task-Review nicht sieht                                                        |
| Microsoft.Extensions.AI + Anthropic/OpenRouter        | KI <b>im Produkt</b> (Classifier, Enrichment, Embeddings)                                                                                                                       |
| Eigene CAS-Skills                                     | <code>cas-aise-todo-list</code> , <code>cas-aise-grade-self-check</code> , <code>sync-ai-instructions</code> — Rubrik-Verankerung, Block-/Phasen-Steuerung, Cross-Project-Reuse |

## 2. Workflow-Entwicklung (Block 1 → Block 5)

Mit jedem Block wurde ad-hoc-Chat gegen strukturierteren, isolierteren Subagent-Dispatch getauscht. Der Mehraufwand für Struktur hat sich durch weniger Mid-Task-Eskalationen und seltenere „Agent-ist-abgedriftet“-Momente bezahlt gemacht.

| Block              | Workflow-Verschiebung                                                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 —<br>Einführung  | Ad-hoc-Chat für Architektur; Copilot inline für Code.                                                                                                                              |
| 2 —<br>Frontend    | Erster sustained CLI-Einsatz mit phasenbasierter Disziplin ( <code>/ui-brainstorm</code> → <code>/ui-flow</code> → <code>/ui-build</code> → <code>/ui-review</code> ).             |
| 3 —<br>Service     | Erster vollständiger <code>brainstorming</code> → <code>writing-plans</code> → <code>subagent-driven-development</code> -Slice; zweistufiges Review (Spec + Quality) pro Task.     |
| 4 —<br>Persistence | Sonnet als Default, Haiku für Mechanik; Per-Task-Quality-Review zugunsten einer Branch-Review am Slice-Ende fallen gelassen — ca. 60 % Token-Burn reduziert ohne Qualitätsverlust. |
| 5 —<br>Deployment  | <code>/ultrareview</code> und rubrik-gegrundetes <code>cas-aise-grade-self-check</code> ergänzt; jede Block-Abschluss-Prüfung läuft durch die Skill.                               |

*Subagent-Driven Development (SDD)* bezeichnet das hier verwendete Muster: ein Plan wird in kleine, wohldefinierte Tasks zerlegt, jeder Task von einem isolierten Implementer-Subagent umgesetzt, während **getrennte** Subagents Spezifikation und Code-Qualität prüfen.

Bis Block 5 verantworten solche Implementer-Subagents ganze Slices end-to-end. Die menschliche Arbeit konzentriert sich dadurch auf zwei Stellen: **Spec** (wo die Verträge festgelegt werden) und **Review** (wo die System-Invarianten geprüft werden). Anders gesagt: Sobald die KI das Implementieren übernimmt, ist nicht mehr die Tipp- bzw. Implementierungsgeschwindigkeit der begrenzende Faktor, sondern die Qualität von Spezifikation und Review. Der **Engpass** des Projekts —

die Stelle, die das Tempo bestimmt — verschiebt sich also vom Code-Schreiben hin zu diesen beiden Tätigkeiten.

---

### 3. Was funktioniert hat

---

1. **Der Plan ist der Vertrag.** Enthält der Plan exakte Pfade, exakten Code, exakte Commit-Messages und exakte Verifikationskommandos, operiert der Implementer mit engem Spielraum und meldet DONE statt NEEDS\_CONTEXT. Die 95 %+ „AI-drafted“-Anteile im Produktionscode sind nur deshalb erreichbar.
  2. **Review-Kadenz muss zur Fehlerklasse passen.** Per-Task-Spec + Per-Task-Quality war für den ersten SDD-Lauf (Block 3) richtig. Ab Block 4 fing Per-Task-Quality nichts Neues mehr, während ein branchweites `/ultrareview` am Slice-Ende genau die Architektur-Fehler fand, die Per-Task-Review nicht sehen konnte.
  3. **Eigene Skills als Memory-Layer des Projekts.** `cas-aise-grade-self-check`, `cas-aise-todo-list`, `sync-ai-instructions` decken Concerns ab, die das Upstream-Plugin nicht behandelt: Rubrik-Verankerung, kalendergetriebene Priorisierung, Cross-Project-Reuse. Ohne `cas-aise-grade-self-check` driftete die Rubrik-Abdeckung in langen Sessions.
  4. **KI für rigide Struktur-Artefakte, Mensch für architektonisches Urteil.** GitHub-Actions-YAML, ADR-Scaffolds, ProblemDetails, EF-Migrationen — rigide Strukturen, in denen der 90 %-korrekte AI-Erstwurf reale Zeit spart. Architektur-Entscheidungen (hexagonaler Split, In-Process vs. RabbitMQ) bleiben beim Menschen; die KI listet Alternativen, sie wählt nicht.
  5. **Konkrete Korrektur-Geschichten sind die überzeugendste Evidenz für den KI-Einsatz.** Eine nachvollziehbare Kette — *die KI erzeugte Code X, der Smoke-Test fand den Fehler Y, der Fix landete in Commit Z* — macht den KI-gestützten Workflow überprüfbar (Defekt, fixender Commit, Lehre sind verlinkt). Pauschale Aussagen wie „die KI hat geholfen“ leisten das nicht und sind für eine Bewertung wertlos.
- 

### 4. Wiederkehrende Fehlerquellen

---

1. **Single-Pass-AI-Review übersieht System-Invarianten.** Block-5-Beispiel: AI-generierter Code, der Embedding-Generation in

`EfCaptureService.SubmitAsync` integrierte, war lokal korrekt (eine Transaktion), global aber falsch — ein langsamer Provider sprengt das Submit-Latenz-Budget. Per-Task-Quality-Review fand das nicht; `/ultrareview` fand es beim ersten Lauf.

## 2. **Deployment-Shape-Failures sind systematisch unter-getestet.**

Compose- `${X:-}` -Interpolation substituiert leere Strings (nicht null), wodurch `??` -Defaults still no-op'en; `.editorconfig` fehlte im Docker-Build-Kontext; ein Casing-Mismatch ( `EMBEDDINGS__APIKEY` vs. `Embeddings__ApiKey` ) schaltete ein ganzes Feature aus. Fünf solcher Defekte fing `just smoke-prod` an einem Nachmittag.

## 3. **Training-Data-Lag bei aktuellen Libraries.** `Pgvector.EntityFrameworkCore`

wurde als in-the-box-Feature angenommen (ist ein separates Paket); `MassTransit.Testing` umgekehrt als separates Paket (ist Teil der Haupt-Assembly). API-Behauptungen der KI müssen gegen das installierte Paket verifiziert werden.

## 4. **Offene Prompts inflationieren den Scope.** „Füge X hinzu“ ohne Plan produziert eine Feature-Suite. Gegenmittel: kleinteilige, TDD-geordnete Tasks mit Inline-Code, exakten Pfaden und Commit-Messages.

## 5. **Pläne aus unvollständigem Repo-Wissen.** Ein Slice-Plan übersah eine bestehende Test-Datei und vorhandene Enum-Werte; seitdem liest Planning jede referenzierte Datei, *bevor* der Plan eingefroren wird.

---

# 5. Tragfähige Disziplinen (über das Projekt hinaus)

---

Die wertvollsten Erkenntnisse sind weniger einzelne Tools als wiederholbare Arbeitsweisen:

- **AI-Instructions wie ein Onboarding-Dokument pflegen.** `CLAUDE.md` für harte Regeln, `.ai/` -Instructions als kanonische Konventionsreferenz. Ohne diese Schichten driftet jeder Agent in seine Defaults; mit ihnen werden Instruktionen einmal geschrieben und von Claude Code, Codex, Copilot gleichermassen befolgt.
- **Eigene Skills schreiben.** Kleine, scharf umrissene Markdown-Definitionen mit Trigger-Description und geprüftem Vorgehen — der Agent erkennt selbst, *wann* ein Skill anzuwenden ist, und folgt einer Checkliste statt zu improvisieren.

- **Skills thematisch in Plugins aufteilen.** Jede Skill-Description kostet System-Prompt-Tokens und verwässert die Trigger-Schärfe. Nur das Plugin aktivieren, das zur aktuellen Arbeit passt.
  - **Context-Hygiene: Logs via Datei.** Console-/Build-/Test-Output in eine Datei schreiben und gezielt mit `Read / grep` lesen, statt ganze Streams ins Kontextfenster zu kippen — subjektiv Faktor 5–10 weniger Token pro Debug-Session.
  - **Code-Exploration bewusst eskalieren.** Erst `rg` (ripgrep, schnelle Text-Suche); wenn drei Suchanläufe nichts liefern, ist die Frage semantisch und kein reines Text-Matching. Dann helfen ein **LSP** (Language Server Protocol — löst Symbole, Referenzen und Definitionen auf, statt nur Textstellen zu finden) oder semantische Such-Tools.
  - **Spec → Plan → Implement mit harten `/clear` -Schnitten.** Jede Phase produziert ein Artefakt auf Disk, das die nächste Phase als reinen Input zurücklikt; der vorherige Dialog wird verworfen. Das ist die strukturelle Variante des Logs-via-Datei-Patterns.
- 

## 6. Was ich anders machen würde

---

- **Rubrik-Verankerung ab Block 1, nicht Block 3.** `cas-aise-grade-self-check` entstand mittendrin; Block 1–2 hatten keine formalen `use-cases.md` / `nfa.md` / `acceptance-criteria.md`. Retroaktive Doku-Arbeit kostete Zeit.
  - **`/ultrareview` ab Block 3, nicht Block 5.** Branchweites Multi-Agent-Review fängt die Fehlerklasse, die Per-Task-Review nicht sieht — günstiger nach Slice B als nach einem 21-Task-MVP.
  - **Ein `just smoke` -Rezept pro Block, nicht nur Block 5.** Ein einfaches Smoke (App booten, Feature-Pfad curlen) hätte mindestens drei der fünf späten Bugs früher gefangen.
  - **Mechanische Patterns ins Plan-Template kodifizieren.** Beispiel `InternalsVisibleTo`: ein .NET-Attribut, das einem Test-Projekt Zugriff auf die `internal`-Typen des Produktiv-Projekts gibt, ohne diese öffentlich (`public`) zu machen — die Domänen-Entities bleiben gekapselt und sind trotzdem testbar. Implementer-Subagents weiteten bei Compile-Fehlern stattdessen oft reflexartig die Sichtbarkeit auf `public` aus; einmal im Plan-Template ausgeschrieben (ebenso das EF-Core-Test-Host-Setup), verschwindet diese Friktion.
-

## 7. Hat die Ausgangshypothese gestimmt?

---

Die Hypothese zu Projektbeginn (Block 1) lautete: Claude sei „ein schneller Tipper mit guter Library-Kenntnis“ — also nützlich für Boilerplate, fragwürdig bei Architektur, und insgesamt ein Produktivitäts-Multiplikator, der den Workflow aber nicht grundlegend verändert. Bis Block 5 erwies sich diese Annahme in **beiden** Punkten als falsch — und zwar in entgegengesetzte Richtungen.

**Beim Boilerplate wurde die KI unterschätzt.** Über 95 % des Produktionscodes sind KI-erstellt. Das heisst aber nicht „die KI hat 95 % der Arbeit gemacht“: die verbleibenden ~5 % menschlicher Anteil verlagern sich fast vollständig auf Entscheidungen mit hohem Urteilsbedarf — Scope, Verträge, Trade-offs. Das reine Tippen ist nur noch ein kleiner Bruchteil der aufgewendeten Zeit.

**Bei der Architektur wurde dagegen die menschliche Rolle unterschätzt.** Der Embedding-on-Submit-Fall (Kapitel 4) zeigt es: lokal korrekter, plausibel aussehender Code, der eine System-Invariante — das Submit-Latenz-Budget — verletzte und ein oberflächliches Single-Pass-Review passiert hätte. Solche Fehler produziert die KI schneller, als ein flüchtiges Review sie fangen kann. Die menschliche Rolle an der Architektur-Grenze ist deshalb nicht geschrumpft, sondern **gewachsen**: Gerade weil der Agent genug korrekten Code schnell genug liefert, wird der entscheidende menschliche Beitrag der **Filter** aus Design und Review.

Beide Beobachtungen zeigen in dieselbe Richtung und führen direkt zum Fazit.

---

## 8. Fazit

---

KI hat keine Rolle im Workflow ersetzt — sie hat **verschoben, welche Rolle der Engpass ist**. Implementierungs-Durchsatz ist nicht mehr die Beschränkung; Design- und Review-Durchsatz sind es. Für das nächste Projekt: zuerst die Spec-Dokumente, `/ultrareview` ab Tag eins, ein Smoke-Target vor dem ersten Feature.

### Drei Veto-Entscheidungen («nie an die KI delegiert»)

Drei Klassen von Entscheidungen blieben bewusst beim Menschen — die KI durfte Optionen auflisten, aber nicht entscheiden:

1. **Architektur-Stil und Transport.** Die Wahl *modularer Monolith statt Microservices* und *In-Process-MassTransit (In-Memory in Dev, RabbitMQ in Prod)* statt *verteiltem Service-Mesh* traf ich selbst. Die KI listete in den ADRs die Alternativen auf („Alternatives considered“), die Abwägung und Wahl gehörten mir. *Begründung:* Architektur-Trade-offs prägen den Code über Monate und hängen an Kontext (Skala, Betrieb, Homelab), den ein Single-Pass-Vorschlag nicht gewichtet. *Belegt:* ADR 0001, ADR 0002 (§ „Alternatives considered“).
2. **System-Invarianten gegen lokal-korrekten KI-Code.** Die KI erzeugte Code, der die Embedding-Generierung in `EfCaptureService.SubmitAsync` integrierte — lokal korrekt (eine Transaktion), global falsch, weil ein langsamer Embedding-Provider das Submit-Latenz-Budget gesprengt hätte. Ich verwarf das und verschob die Embedding-Erzeugung in den asynchronen `CaptureEmbeddingConsumer`. *Begründung:* System-Invarianten (Latenz, Konsistenzgrenzen) sind genau die Fehlerklasse, die ein lokal fokussiertes Review nicht sieht. *Belegt:* `docs/ai-usage.md` („Embedding-on-Submit“), `docs/insights/block-5.md` + fixender Commit.
3. **Scope und Port-Verträge.** Was gebaut wird (welche Skills/Integrationen) und die Form der treibenden/getriebenen Ports ( `IClassifier` , `ISkillIntegration` ) blieben menschlich festgelegt; offene Prompts blähen den Scope auf, deshalb wurde jeder Plan vor der Umsetzung von mir eingefroren („der Plan ist der Vertrag“). *Begründung:* Der Vertrag zwischen Modulen ist die teuerste spätere Änderung — er gehört vor die Generierung, nicht in sie. *Belegt:* `docs/spec/` (Port-/UC-Definitionen), die Pläne unter `docs/superpowers/plans/`.

Der Übertrag auf die künftige Arbeitsweise: Diese drei Grenzen — Architektur, System-Invarianten, Verträge — bleiben auch im nächsten Projekt menschlich; die KI beschleunigt das Davor (Recherche, Optionen) und das Danach (Generierung, Review), nicht die Entscheidung selbst.

## Transfer CAS → Projektarbeit

| CAS-Block | Mitgenommener Inhalt          | Konkrete FlowHub-Entscheidung                                                  |
|-----------|-------------------------------|--------------------------------------------------------------------------------|
| 1 — Einfü | Architekturoptionen begründet | <b>Modular Monolith mit hexagonaler Schichtung</b> (ADR 0001) + die ADR-Praxis |

| CAS-Block       | Mitgenommener Inhalt                           | Konkrete FlowHub-Entscheidung                                                                                                                                                         |
|-----------------|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hrung           | abwägen; KI-Tooling aufsetzen                  | selbst; Tooling-Fundament ( <code>CLAUDE.md</code> , <code>.ai/</code> , eigene Skills)                                                                                               |
| 2 — Frontend    | Render-Modelle, Frontend-Architektur           | <b>Blazor Interactive Server</b> statt WASM-SPA (ADR 0001); vierstufige UI-Pipeline, die Entwurf vor Code erzwingt                                                                    |
| 3 — Service     | Service-Architekturen, Protokolle, KI-Services | <b>Asynchrone In-Process-Pipeline</b> (RabbitMQ + MassTransit, ADR 0002/0003) statt synchronem Microservice-Geflecht; KI-Klassifikation als erster „intelligenter Service" (ADR 0004) |
| 4 — Persistence | Persistenzform wählen, ORM-Abstraktion         | <b>EF Core + PostgreSQL</b> mit Repository-Abstraktion (ADR 0005); KI-Suche über <b>pgvector auf derselben Postgres</b> (ADR 0006)                                                    |
| 5 — Deployment  | Containerisierung, CI/CD, Observability        | Compose-Stack, drei GitHub-Actions-Workflows, <b>OpenTelemetry + Prometheus + Grafana</b> ; Policy-ADRs 0007-0009                                                                     |

Das eigentliche Querschnittsthema des CAS — **KI-assistierte Software-Entwicklung** — war zugleich die Arbeitsweise der gesamten Projektarbeit. Der direkteste Transfer sind die oben beschriebenen Disziplinen: gepflegte Agent-Instructions, eigene Skills, Context-Hygiene über Datei-Artefakte und der Spec→Plan→Implement-Rhythmus.

---

*Reflexion, Juni 2026. Erstellt mit Unterstützung von Claude (Anthropic) gemäss den FFHS-Richtlinien für KI-Einsatz in Projektarbeiten. Volltext der blockweisen KI-Nutzung: `docs/ai-usage.md` .*